

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 — Introduction to Artificial Intelligence

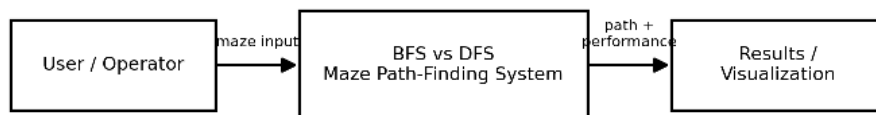
PRN	25UAM08	Name	Prathmesh Kore
Division	B	Date	22/09/2026
GitHub / Portfolio Link	github.com/prathmeshkore13-collab/IAI	System	BFS vs DFS Maze Path-Finding

1 System Title & Short Description

The BFS vs DFS Maze Path-Finding System generates solvable grid mazes and searches for a path from the start cell to the goal cell. It compares Breadth-First Search (BFS) and depth-limited Depth-First Search (DFS). The system records path length, nodes expanded and execution time for different maze sizes (20x20, 40x40, 70x70). It also supports profiling with py-spy and produces comparison charts. This architecture continues the same search/maze system used in SLE-1 and SLE-2.

2. Context Diagram (Level 1)

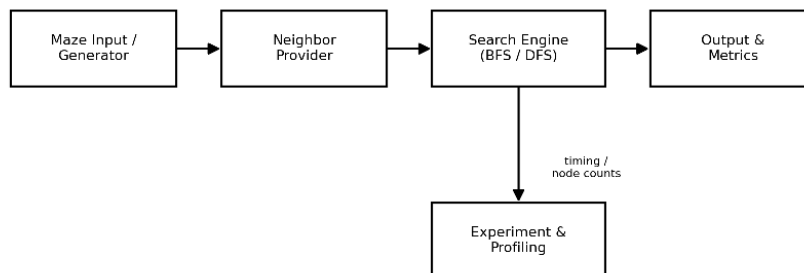
C4 Level 1 — Context Diagram



The user provides a maze size and chooses which algorithm to run. The system generates a solvable maze, applies BFS or DFS, and returns a path together with performance information such as nodes expanded and runtime. The output can also be visualized through generated charts and flame graphs.

3. Container Diagram (Level 2)

C4 Level 2 — Container Diagram



Container	Responsibility
Maze Input / Generator	Creates a random grid maze, sets start and goal, and regenerates until the maze is solvable.
Neighbor Provider	Returns valid non-wall neighbouring cells in the fixed order: up, down, left, right.
Search Engine (BFS / DFS)	Runs BFS with a queue or DFS with a stack and depth limit; tracks visited cells and carries the path directly with each queued/stacked node.

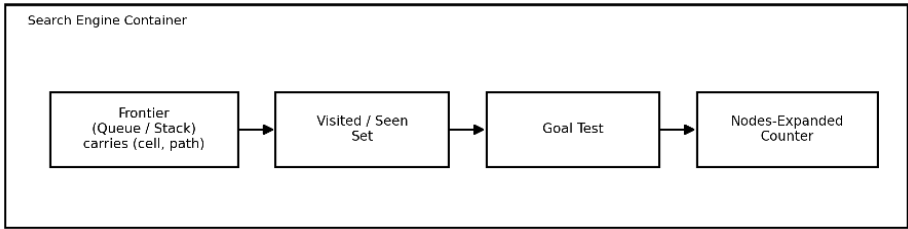
Output & Metrics	Reports path length, nodes expanded and timing results for each maze size tested.
Experiment & Profiling	Runs repeated tests for 20x20, 40x40 and 70x70 mazes and supports py-spy CPU profiling.

4. Component Diagram (Level 3)

The main container selected for detailed decomposition is the Search Engine.

C4 Level 3 — Components inside Search Engine

BFS uses a queue + path-in-node; DFS uses a stack + path-in-node, depth-limited



Its frontier stores (cell, path) pairs waiting to be explored, while the visited/seen set prevents repeated exploration. The goal test checks whether the current cell is the target cell. Unlike a parent-map approach, this implementation carries the path-so-far directly inside each queued/stacked item, so no separate path-reconstruction step is needed once the goal is found. BFS uses a queue (first-in-first-out), whereas DFS uses a stack (last-in-first-out) with a depth limit.

5. Code Level Overview (Level 4)

- generate_maze(rows, cols, wall_probability): creates a solvable grid maze with start and goal cells.
- get_neighbors(maze, cell): returns valid neighbouring cells in a fixed order (up, down, left, right).
- bfs(maze, start, goal): performs breadth-first search, returning (path, nodes_expanded).
- dfs(maze, start, goal, depth_limit): performs depth-limited depth-first search, returning (path, nodes_expanded).
- run_test(rows, cols): times both algorithms on one maze and prints path length, nodes expanded and execution time.

6. Design Decisions

The architecture is kept modular so maze generation, neighbour handling, search, and performance measurement each have clear responsibilities. BFS and DFS share the same maze and neighbour logic, which makes their comparison fair. A fixed neighbour order (up, down, left, right) is retained because it directly affects DFS's behaviour and path choice. Rather than using a separate parent/came-from map, the path is carried directly inside each frontier item – a simpler design for this scale of maze, at the cost of using slightly more memory per queued/stacked node since each one stores its own full path copy.

7. AI Contribution Note

- AI tools used: Claude (Anthropic), as recorded in the SLE-1 and SLE-2 Contribution Logs.
- What AI helped with: Drawing the three C4 diagrams based on the real structure of my maze_bfs_dfs.py code, and structuring this report in the required template format.
- What I did myself: Wrote and ran the actual BFS/DFS maze code myself; decided

which containers and components genuinely matched my implementation (for example, describing the Frontier as carrying (cell, path) pairs rather than a parent map, since that is how my code actually works); checked every box in the diagrams against my real functions before accepting them.

8. Conclusion

This SLE-3 helped convert the SLE-1/SLE-2 implementation into a complete software architecture view. The C4 model shows the system from the user-level context down to containers, search components, and individual functions. I learned that separating maze generation, search, and performance measurement makes the system easier to understand and extend, and that an architecture diagram is only useful if it actually matches the real code — which is why each box here was checked against my own functions rather than copied from a generic template.